

StockEasy

Decisões de Produto

2026-05 · DESAFIO TÉCNICO

StockEasy — Decisões de Produto

Visão

O **StockEasy** é uma aplicação fullstack de gestão de produtos e movimentações de estoque (entradas e saídas) entregue como parte de um processo seletivo técnico. O backend é uma **ASP.NET Core Web API (.NET 8 LTS + Dapper + PostgreSQL 16)** e o frontend é uma **SPA Vue 3 + TypeScript estrito + Tailwind**, com regras de negócio canônicas como validação de saldo insuficiente, idempotência obrigatória em mutações sensíveis e histórico imutável de movimentos.

O valor central é demonstrar competência fullstack através de uma implementação **limpa, testada, organizada e agentic-ready** que cumpre integralmente o spec do desafio — porque o objetivo final é passar na avaliação técnica. Visualmente, o produto é parte da família **RoboteAsy** (paleta azul corporativa, primary **#1863DC**, tipografia Inter).

Contexto

Esta entrega é um **deliverable de avaliação técnica** com prazo de 1 a 3 dias e formato fork-PR no GitHub. Os critérios de avaliação explicitamente listados no desafio são:

- Organização do código (separação clara backend/frontend)
- Boas práticas (service layers, DTOs, controllers)
- Validações no backend E no frontend
- Clareza na estrutura de componentes Vue
- UX/UI básica mas funcional
- Tratamento de erros amigáveis

Cada decisão estratégica documentada abaixo mapeia para um ou mais desses critérios. Diferenciais conscientes além do spec — testes (xUnit + Vitest), docker-compose com hot reload, API agentic-friendly, soft delete, 10 heurísticas de Nielsen, documentação PDF branded — foram inclusos sem inflar o escopo do MVP.

Decisões Estratégicas

As decisões abaixo são extraídas verbatim da tabela **Key Decisions** em **.planning/PROJECT.md** (D1..D31). A numeração **D{N}** é estável e referenciada em outras peças da documentação (**02-architecture.md** , **03-business-rules.md**) e nos commits.

D3 — .NET 8 (LTS) sobre .NET 9

Estabilidade e suporte longo são preferidos em avaliações profissionais. .NET 8 LTS tem janela de suporte até novembro de 2026, casa com o projeto de referência **BancoShu** , e roda em

qualquer ambiente moderno sem surpresas. .NET 9 só agregaria features marginais (collection expressions já estão em C# 12) ao custo de menor previsibilidade. Trade-off consciente: estabilidade > novidade.

D5 — PostgreSQL 16 via Dapper (sem EF Core)

A convenção do projeto de referência `BancoShu` é Dapper + SQL raw + `init.sql` versionado, sem migrations. Replicamos o mesmo padrão por três motivos: (1) maturidade técnica que o avaliador vai notar — escrever SQL legível é diferencial; (2) o schema do StockEasy é pequeno e estável (duas tabelas), migrations seriam over-engineering; (3) Dapper é leve, performático e dispensa as abstrações pesadas do EF Core. O `init.sql` é montado no container Postgres via `docker-compose.yml` e roda automaticamente na primeira subida.

D6 — Single-project N-tier (NÃO Clean Architecture)

A arquitetura do backend é uma **pasta única** `Inventory/` com `Controllers/`, `Services/`, `Repositories/`, `Entities/`, `Dtos/`, `Validators/`, `Exceptions/`, `Middleware/` e `Infra/`, mais o projeto irmão `Inventory.Tests/`. **NÃO** dividimos em `Inventory.Domain`, `Inventory.Application`, `Inventory.Infrastructure`. Clean Architecture (4 projetos) seria over-engineering para um deliverable de avaliação com duas entidades — o avaliador valoriza clareza sobre cerimônia. O resultado é uma codebase navegável em segundos: o avaliador abre `Inventory/Services/StockMovementService.cs` e lê a regra de saldo insuficiente sem precisar atravessar fronteiras de projeto.

D7 — Identifiers em inglês, mensagens em PT-BR

Identifiers de código (`Product`, `StockMovement`, `ProductType`, `MovementType`, `IdempotencyKey`, `errorCode`) ficam em **inglês**. Mensagens, hints, labels e copy ao usuário ficam em **português (BR)**. Esta convenção é coerente com `BancoShu`, vale para markdown sources nesta documentação, e respeita o contexto: avaliadores são brasileiros, mas a indústria fala C# / Vue em inglês. Soft constraint: nomes de pasta de feature seguem o inglês do recurso (`features/products/`, `features/stock/`).

D9 — FluentValidation cobre 100% do input validation

Toda validação de input passa por uma classe `*Validator` em `Inventory/Validators/`. Sem `DataAnnotations`. Sem validação manual no controller. Mensagens em PT-BR são espelhadas **byte-a-byte** no frontend pelos schemas Zod (`features/<feat>/schemas/`), de modo que a UX preventiva (`onBlur`) emita exatamente o mesmo texto que o backend (autoridade) emitiria — critério “validações no backend E no frontend” satisfeito sem divergência.

D10 — Exception flow centralizado: DomainException → ExceptionHandlingMiddleware

Toda exceção de domínio herda de `DomainException` (`Inventory/Exceptions/DomainException.cs`) com cinco campos: `ErrorCode` (slug `SCREAMING_SNAKE_CASE`), `Category` (`VALIDATION` / `BUSINESS_RULE` / `NOT_FOUND` / `INTERNAL`), `Hint` (PT-BR dinâmica), `Retryable` (bool) e `Details` (objeto estruturado). Controllers **nunca** fazem try/catch. O `ExceptionHandlingMiddleware` é o único ponto de tradução `Exception → HTTP status + ErrorResponse` . O resultado é um fluxo previsível: throw na regra → middleware → JSON canônico ao cliente — sem ramificações ad-hoc. Hints são construídas no construtor de cada exception tipada com dados reais do contexto (não strings genéricas).

D18 — API agentic-friendly desde o MVP (não chat)

Esta é a decisão silenciosa mais importante do StockEasy. A API v1 já é desenhada para ser consumida por LLMs como **tools** em v2, sem precisar de retrabalho. Quatro propriedades agentic estão ativas no MVP:

1. **OperationIds estáveis camelCase verb-noun** (`createProduct` , `listProducts` , `getProduct` , `deleteProduct` , `createStockMovement` , `listStockMovements` , `getStockMovement`) — viram nomes de tools no chat futuro sem renomeação.
2. **errorCode SCREAMING_SNAKE_CASE com vocabulário fechado** — 9 slugs documentados em `03-business-rules.md` ; o LLM aprende o catálogo uma vez e roteia recovery determinístico.
3. **hint dinâmica construída com dados reais** — não strings hardcoded (“Reduza a quantidade para no máximo 55 ou registre uma entrada antes” em vez de “Saldo insuficiente”). LLM e humano absorvem instrução acionável.
4. **Idempotency-Key em mutations sensíveis** — replay determinístico previne efeito duplo quando o LLM (ou rede instável) retransmite a requisição.

O chat com LLM fica em v2 (ver `## Roadmap v2`). Como diferencial silencioso, o avaliador identifica o cuidado ao abrir `/swagger` e ler uma resposta canônica de erro.

D21 — Idempotency-Key required em POST /api/stock-movements

Todo `POST /api/stock-movements` exige header `Idempotency-Key: <UUID v4>` . Ausência → `400 MISSING_IDEMPOTENCY_KEY` com hint instruindo `crypto.randomUUID()` . Replay com mesma chave → `200 OK` (não `201`) + header `Idempotency-Replay: true` + body byte-idêntico ao da primeira resposta — sem efeito colateral duplicado, mesmo sob race-condition (recovery via Postgres `SqlState 23505` em `idempotency_key UNIQUE`). O frontend injeta o header automaticamente no Axios interceptor (FRONT-12, `crypto.randomUUID()` por submit). `POST /api/products` NÃO exige Idempotency-Key — `code UNIQUE` no banco já dá idempotência natural por domínio.

D24 — Zero N+1 queries

Toda listagem que retorna dados relacionados usa **JOIN** ou **batch fetch** (`WHERE id IN @ids`). Nunca loop com query por item. A listagem de histórico de movimentos (`GET /api/stock-movements`) executa **exatamente 2 SQL statements** por página (um JOIN'd SELECT trazendo `productCode` + `productDescription` , e um COUNT para a paginação) — verificado em produção via `log_statement='all'` (evidência em `.planning/phases/03-stock-movements-vertical-slice/03-VERIFICATION.md` §MOVE-09). O projeto de referência **BancoShu** tem um N+1 em `TransferService.GetHistoryAsync` — **não copiamos esse padrão**.

D25 — Soft delete em produtos via `deleted_at`

Produtos não são hard-deletados. `DELETE /api/products/{id}` faz `UPDATE products SET deleted_at = now() WHERE id = @id AND deleted_at IS NULL` . Listagens filtram `WHERE deleted_at IS NULL` por padrão; `?includeDeleted=true` retorna todos. `GET /api/products/{id}` retorna mesmo se deletado (com `deletedAt` populado) — permite recuperação contextual. **Movimentos de produtos soft-deletados continuam acessíveis no histórico** (auditoria preservada). Códigos de produto soft-deletados **NÃO podem ser reutilizados** — o `UNIQUE` em `products.code` no `init.sql` não filtra por `deleted_at` , o que torna o anti-reuso uma propriedade do schema (PROD-06 enforcement gratuito no nível do banco).

D27 — Aplicar 10 heurísticas de Nielsen

O critério de avaliação "UX/UI básica mas funcional" é endereçado aplicando 10 heurísticas de Nielsen no frontend:

1. **Visibilidade**: 4 estados em toda lista/form (loading skeleton / empty com CTA / error com `apiError.hint` + retry / success com dados).
2. **Mundo real**: formatação BR via `Intl` (`R$ 1.234,56` , `dd/mm/yyyy HH:mm` , separador de milhar) centralizada em `src/shared/format.ts` .
3. **Controle e liberdade**: modais com botão Cancelar à esquerda, `Esc` fecha, foco retorna ao trigger.
4. **Consistência**: botão primário direita, destrutivo `bg-danger` , labels acima de inputs.
5. **Prevenção**: validação `onBlur` inline, submit desabilitado com erros, "Disponível: N unidades" no form de saída.
6. **Reconhecimento sobre lembrança**: searchable dropdown de produto com `code` – `description` .
7. **Minimalista**: 1 ação primária por tela, whitespace generoso.
8. **Recuperação**: toast de erro consome `apiError.hint ?? apiError.message` (a API faz o trabalho da cópia).
9. **A11y básico**: semantic HTML, `<label for>` , `aria-describedby` em erros, focus ring `focus:ring-2 focus:ring-brand-500` , contraste WCAG AA.

D30 — Documentação em docs/ com 3 markdowns + PDFs branded

A documentação final fica em `docs/` na raiz do repositório com três markdowns (`01-product-decisions.md` , `02-architecture.md` , `03-business-rules.md`) e três PDFs branded em `docs/dist/` . A pipeline (Plano 04-05) é **Pandoc + WeasyPrint + mermaid-filter** com CSS Paged Media: paleta StockEasy, tipografia Inter, capa cheia, header com título do documento, footer com paginação. Diagramas Mermaid renderizam como SVG inline. PDFs são commitados no repositório (D31) — o avaliador abre `docs/dist/01-product-decisions.pdf` sem instalar nenhuma toolchain.

Trade-offs Conscientes

Cada item abaixo é uma **fronteira deliberada** — está fora do escopo do MVP por razão registrada:

- **Sem autenticação / login** — não pedido no spec; atrasaria a entrega em 1-3 dias.
- **Sem chat com LLM dentro do app em v1** — planejado para milestone v2. A API v1 entrega o substrato agentic (operationIds, errorCodes, hints, idempotência) para que v2 seja uma camada fina por cima.
- **Sem biblioteca de UI pronta (PrimeVue / Vuetify / Element)** — escolha consciente por Tailwind + 15 primitivos `Base*` próprios. Demonstra capacidade de construir UI do zero (critério de avaliação).
- **Sem EF Core / Migrations** — Dapper + `init.sql` versionado, padrão `BancoShu` (D5).
- **Sem Pinia / Vuex** — composables only (`useProducts` , `useStockMovements`); o estado CRUD é simples e composables bastam.
- **Sem GitHub Actions / CI remoto** — TEST-07 explicitamente diz "CI local" (rodar `dotnet test` e `npm test` na máquina). Workflows seriam scope creep para um deliverable de 1-3 dias.
- **Sem dark mode / i18n / atalhos de teclado / auto-save** — overkill para o MVP; light mode + PT-BR + interação padrão é o suficiente.

Roadmap v2

A v2 do StockEasy (planejada pós-entrega) introduz um **chat com LLM** dentro do app que consome a própria API v1 como tools. Os requisitos `CHAT-*` em `.planning/REQUIREMENTS.md` definem o escopo:

- **CHAT-01:** Endpoint `POST /api/chat` recebe mensagem do usuário, chama um LLM (server-side ou local, decisão pendente) com tools definidas a partir do OpenAPI v1.
- **CHAT-02:** O LLM consome os `operationId` existentes (`listProducts` , `getProduct` , `createProduct` , `registerStockMovement` , `listStockMovements`) **como nomes de tools** — sem retrabalho na API.

- **CHAT-03:** UI de chat no frontend (drawer lateral ou page dedicada, decisão de UX pendente).
- **CHAT-04: Confirmação humana obrigatória** antes de executar mutations destrutivas ou irreversíveis (`Outbound` , `deleteProduct`) — modal CONF-01 / CONF-02 reaproveitado.
- **CHAT-05:** Catálogo inicial de tools = `listProducts` , `getProduct` , `createProduct` , `registerStockMovement` , `listStockMovements` (sem `deleteProduct` na v2.0 — risco de regressão prefere acima de pressa).

A v1 foi desenhada **agentic-ready precisamente para que v2 seja uma camada fina por cima da API**: `operationIds` estáveis, `errorCodes` estruturados com hints dinâmicas (recovery automático), idempotência em mutations (replay seguro sob retransmissão), enums serializados como string. Nada disso precisa ser retrabalhado.

Sumário

Cada decisão estratégica do StockEasy mapeia para um dos critérios de avaliação explícitos do desafio: **organização do código** (D6 single-project, D7 EN/PT, D10 exception flow); **boas práticas** (D5 Dapper + `init.sql`, D9 `FluentValidation`, D24 zero N+1); **validações em ambos lados** (D9 com schemas `Zod` espelhados byte-a-byte); **clareza de componentes Vue** (composables + feature-based folders + `Base*` primitives); **UX/UI básica mas funcional** (D27 10 heurísticas de Nielsen aplicadas); **tratamento de erros amigáveis** (D10 + D18 + D20 hints dinâmicas com dados reais consumidas pelo toast no frontend).

Os diferenciais conscientes — testes (xUnit + Vitest), docker-compose com hot reload, API agentic-friendly desde o MVP, soft delete, idempotência transparente, documentação PDF branded — são entregues sem inflar o escopo do MVP. Cada um é uma resposta a uma maturidade técnica que o avaliador deve perceber sem que o código precise gritar: o objetivo é passar na avaliação técnica entregando código que demonstre intencionalidade em cada camada.